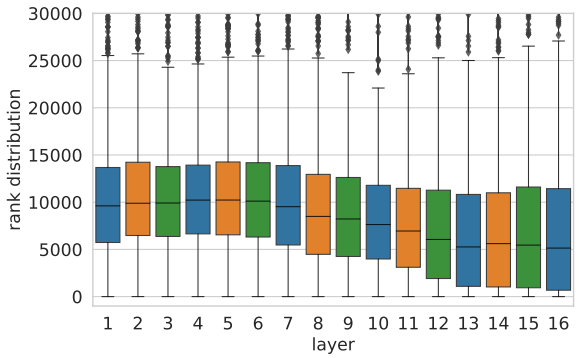

Figure 5: Distribution of the rank of the next-token in the top-1 trigger example of $\mathbf{k}_i^\ell$ (w_i^ℓ), according to the ranking induced by the value vector $\mathbf{v}_i^\ell$. We cut the tail of the distribution, which stretches up to the vocabulary size (∼270K tokens).

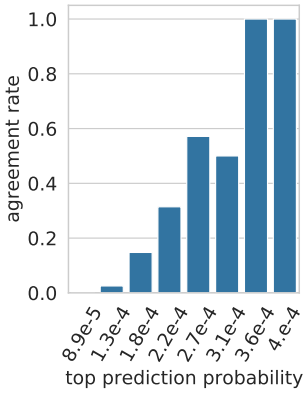

Figure 6: Agreement rate (between the top-ranked token based on the value vector $\mathbf{v}_i^\ell$ and the next token of the top-ranked trigger example associated with the key vector $\mathbf{k}_i^\ell$) as a function of the maximal probability assigned by the value vector.

agreement rate is close to zero in the lower layers (1-10), but starting from layer 11, the agreement rate quickly rises until 3.5%, showing higher agreement between keys and values on the identity of the top-ranked token. Importantly, this value is orders of magnitude higher than a random token prediction from the vocabulary, which would produce a far lower agreement rate (0.0004%), showing that upper-layer memories manifest non-trivial predictive power.

Next, we take the next token of $\mathbf{k}_i^\ell$'s top-1 trigger example (w_i^ℓ), and find where it ranks in the value vector's distribution $\mathbf{p}_i^\ell$. Figure 5 shows that the rank of the next token of a trigger example increases through the layers, meaning that w_i^ℓ tends to get higher probability in the upper layers.

Detecting predictive values. To examine if we can automatically detect values with high agreement rate, we analyze the probability of the values' top prediction, i.e., ($\max(\mathbf{p}_i^\ell)$). Figure 6 shows that although these distributions are not calibrated, distributions with higher maximum probabilities are more likely to agree with their key's top trigger example. We then take the 100 values with highest probability across all layers and dimensions (97 out of the 100 are in the upper layers, 11-16), and for each value $\mathbf{v}_i^\ell$, analyze the top-50 trigger examples of $\mathbf{k}_i^\ell$. We find that in almost half of the values (46 out of 100), there is at least one trigger example that agrees with the value's top prediction. Examples are provided in Table 2.

Discussion. When viewed as distributions over the output vocabulary, values in the upper layers tend to assign higher probability to the next-token of examples triggering the corresponding keys. This suggests that memory cells often store information on how to directly predict the output (the distribution of the next word) from the input (patterns in the prefix). Conversely, the lower layers do not exhibit such clear correlation between the keys' patterns and the corresponding values' distributions. A possible explanation is that the lower layers do not operate in the same embedding space, and therefore, projecting values onto the vocabulary using the output embeddings does not produce distributions that follow the trigger examples. However, our results imply that some intermediate layers *do* operate in the same or similar space to upper layers (exhibiting some agreement), which in itself is non-trivial. We leave further exploration of this phenomenon to future work.

5 Aggregating Memories

So far, our discussion has been about the function of a single memory cell in feed-forward layers. How does the information from *multiple* cells in multiple *layers* aggregate to form a model-wide prediction? We show that every feed-forward layer combines multiple memories to produce a distribution that is qualitatively different from each of its component memories' value distributions (Section 5.1). These layer-wise distributions are then combined via residual connections in a refinement process, where each feed-forward layer updates the residual's distribution to finally form the model's output (Section 5.2).

Value	Prediction	Precision@50	Trigger example
$\mathbf{v}^{15}_{222}$	*each*	68%	*But when bees and wasps resemble each*
$\mathbf{v}^{16}_{752}$	*played*	16%	*Her first role was in Vijay Lalwani's psychological thriller Karthik Calling Karthik, where Padukone was cast as the supportive girlfriend of a depressed man (played*
$\mathbf{v}^{13}_{2601}$	*extratropical*	4%	*Most of the winter precipitation is the result of synoptic scale, low pressure weather systems (large scale storms such as extratropical*
$\mathbf{v}^{15}_{881}$	*part*	92%	*Comet served only briefly with the fleet, owing in large part*
$\mathbf{v}^{16}_{2070}$	*line*	84%	*Sailing from Lorient in October 1805 with one ship of the line*
$\mathbf{v}^{12}_{3186}$	*jail*	4%	*On May 11, 2011, four days after scoring 6 touchdowns for the Slaughter, Grady was sentenced to twenty days in jail*

Table 2: Example values, their top prediction, the fraction of their key's top-50 trigger examples that agree with their prediction, and a matching trigger example (with the target token marked in blue).

5.1 Intra-Layer Memory Composition

The feed-forward layer's output can be defined as the sum of value vectors weighted by their memory coefficients, plus a bias term:

$$\mathbf{y}^\ell = \sum_i \text{ReLU}(\mathbf{x}^\ell \cdot \mathbf{k}^\ell_i) \cdot \mathbf{v}^\ell_i + \mathbf{b}^\ell.$$

If each value vector $\mathbf{v}^\ell_i$ contains information about the target token's distribution, how is this information aggregated into a single output distribution? To find out, we analyze the behavior of 4,000 randomly-sampled prefixes from the validation set. Here, the validation set is used (rather than the training set used to find trigger examples) since we are trying to characterize the model's behavior at inference time, not find the examples it "memorizes" during training.

We first measure the fraction of "active" memories (cells with a non-zero coefficient). Figure 7 shows that a typical example triggers hundreds of memories per layer (10%-50% of 4096 dimensions), but the majority of cells remain inactive. Interestingly, the number of active memories drops towards layer 10, which is the same layer in which semantic patterns become more prevalent than shallow patterns, according to expert annotations (see Section 3, Figure 2).

While there are cases where a single memory cell dominates the output of a layer, the majority of outputs are clearly compositional. We count the number of instances where the feed-forward layer's top prediction is *different* from all of the memories' top predictions. Formally, we denote:

$$\text{top}(\mathbf{h}) = \text{argmax}(\mathbf{h} \cdot E)$$

as a generic shorthand for the top prediction from the vocabulary distribution induced by the vector

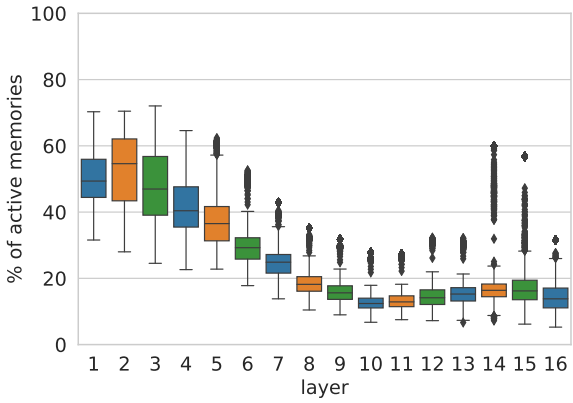

Figure 7: The fraction of active memories (i.e., with positive memory coefficient) out of 4096 memories in every layer, for a random sample of 4,000 examples.

$\mathbf{h}$, and compute the number of examples where the following condition holds:

$$\forall i : \text{top}(\mathbf{v}^\ell_i) \neq \text{top}(\mathbf{y}^\ell)$$

Figure 8 shows that, for any layer in the network, the layer's final prediction is different than *every one* of the memories' predictions in at least ∼68% of the examples. Even in the upper layers, where the memories' values are more correlated with the output space (Section 4), the layer-level prediction is typically not the result of a single dominant memory cell, but a composition of multiple memories.

We further analyze cases where at least one memory cell agrees with the layer's prediction, and find that (a) in 60% of the examples the target token is a common stop word in the vocabulary (e.g. *"the"* or *"of"*), and (b) in 43% of the cases the input prefix has less than 5 tokens. This suggests that very common patterns in the training data might

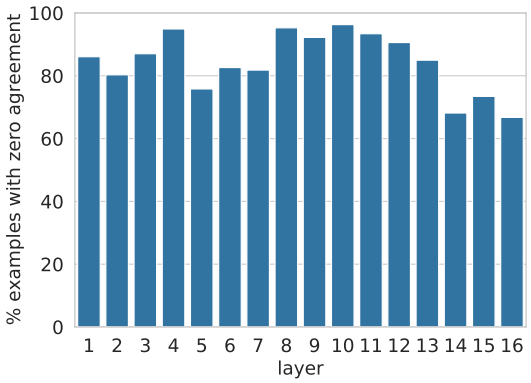

Figure 8: The fraction of examples in a random sample of 4,000 examples where the layer's prediction is different from the prediction of all of its memories.

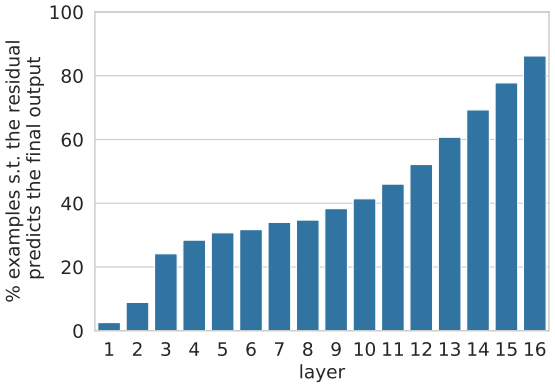

Figure 9: Fraction of examples in each layer, where the residual's top prediction matches the model's output.

be "cached" in individual memory cells, and do not require compositionality.

5.2 Inter-Layer Prediction Refinement

While a single feed-forward layer composes its memories in parallel, a multi-layer model uses the residual connection $\mathbf{r}$ to *sequentially* compose predictions to produce the model's final output:[5]

$$\mathbf{x}^\ell = \text{LayerNorm}(\mathbf{r}^\ell)$$
$$\mathbf{y}^\ell = \text{FF}(\mathbf{x}^\ell)$$
$$\mathbf{o}^\ell = \mathbf{y}^\ell + \mathbf{r}^\ell$$

We hypothesize that the model uses the sequential composition apparatus as a means to *refine* its prediction from layer to layer, often deciding what the prediction will be at one of the lower layers.

To test our hypothesis, we first measure how often the probability distribution induced by the residual vector $\mathbf{r}^\ell$ matches the model's final output $\mathbf{o}^L$ (L being the total number of layers):

$$\text{top}(\mathbf{r}^\ell) = \text{top}(\mathbf{o}^L)$$

Figure 9 shows that roughly a third of the model's predictions are determined in the bottom few layers. This number grows rapidly from layer 10 onwards, implying that the majority of "hard" decisions occur before the final layer.

We also measure the probability mass p that each layer's residual vector $\mathbf{r}^\ell$ assigns to the model's final prediction:

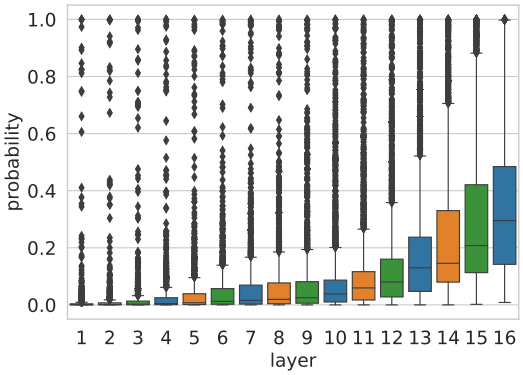

Figure 10: Probability of the token output by the model according to the residual of each layer.

$$w = \text{top}(\mathbf{o}^L)$$
$$\mathbf{p} = \text{softmax}(\mathbf{r}^\ell \cdot E)$$
$$p = \mathbf{p}_w$$

Figure 10 shows a similar trend, but emphasizes that it is not only the top prediction's identity that is refined as we progress through the layers, it is also the model's confidence in its decision.

To better understand how the refinement process works at each layer, we measure how often the residual's top prediction changes following its interaction with the feed-forward layer ($\text{top}(\mathbf{r}^\ell) \neq \text{top}(\mathbf{o}^\ell)$), and whether this change results from the feed-forward layer overriding the residual ($\text{top}(\mathbf{o}^\ell) = \text{top}(\mathbf{y}^\ell)$) or from a true composition ($\text{top}(\mathbf{r}^\ell) \neq \text{top}(\mathbf{o}^\ell) \neq \text{top}(\mathbf{y}^\ell)$).

Figure 11 shows the breakdown of different cases per layer. In the vast majority of examples, the residual's top prediction ends up being the

[5] The residual propagates information from previous layers, including the transformer's self-attention layers.

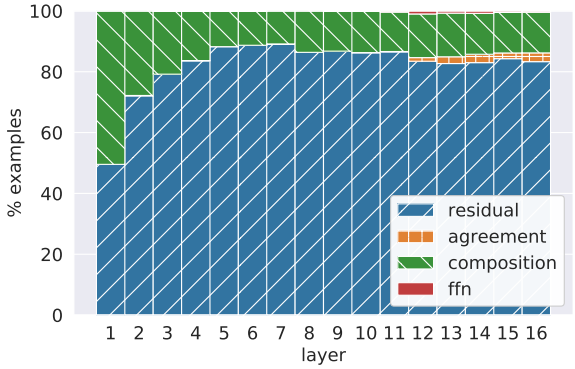

Figure 11: Breakdown of examples by prediction cases: the layer's output prediction matches the residual's prediction (*residual*), matches the feed-forward layer's prediction (*ffn*), matches both of the predictions (*agreement*), or none of the predictions (*composition*). By construction, there are no cases where the residual's prediction matches the feed-forward layer's prediction and does not match the output's prediction.

model's prediction (*residual+agreement*). In most of these cases, the feed forward layer predicts something different (*residual*). Perhaps surprisingly, when the residual's prediction does change (*composition+ffn*), it rarely changes to the feed-forward layer's prediction (*ffn*). Instead, we observe that composing the residual's distribution with that of the feed-forward layer produces a "compromise" prediction, which is equal to neither (*composition*). This behavior is similar to the intra-layer composition we observe in Section 5.1. A possible conjecture is that the feed-forward layer acts as an elimination mechanism to "veto" the top prediction in the residual, and thus shifts probability mass towards one of the other candidate predictions in the head of the residual's distribution.

Finally, we manually analyze 100 random cases of last-layer composition, where the feed-forward layer modifies the residual output in the *final* layer. We find that in most cases (66 examples), the output changes to a semantically distant word (e.g., *"people"* $\rightarrow$ *"same"*) and in the rest of the cases (34 examples), the feed-forward layer's output shifts the residual prediction to a related word (e.g. *"later"* $\rightarrow$ *"earlier"* and *"gastric"* $\rightarrow$ *"stomach"*). This suggests that feed-forward layers tune the residual predictions at varying granularity, even in the last layer of the model.

6 Related Work

Considerable attention has been given to demystifying the operation of neural NLP models. An extensive line of work targeted neuron functionality in general, extracting the properties that neurons and subsets of neurons capture (Durrani et al., 2020; Dalvi et al., 2019; Rethmeier et al., 2020; Mu and Andreas, 2020; Vig et al., 2020), regardless of the model architecture or neurons' position in it. Jacovi et al. (2018) analyzed CNN architectures in text classification and showed that they extract key n-grams from the inputs.

The study of the transformer architecture has focused on the role and function of self-attention layers (Voita et al., 2019; Clark et al., 2019; Vig and Belinkov, 2019) and on inter-layer differences (i.e. lower vs. upper layers) (Tenney et al., 2019; Jawahar et al., 2019). Previous work also highlighted the importance of feed-forward layers in transformers (Press et al., 2020; Pulugundla et al., 2021; Xu et al., 2020). Still, to date, the role of feed-forward layers remains under-explored.

Also related are interpretability methods that explain predictions (Han et al., 2020; Wiegreffe and Pinter, 2019), however, our focus is entirely different: we do not interpret individual predictions, but aim to understand the mechanism of transformers.

Characterizing the functionality of memory cells based on examples that trigger maximal activations has been used previously in NLP (Rethmeier et al., 2020) and vision (Erhan et al., 2009).

7 Discussion and Conclusion

Understanding how and why transformers work is crucial to many aspects of modern NLP, including model interpretability, data security, and development of better models. Feed-forward layers account for most of a transformer's parameters, yet little is known about their function in the network.

In this work, we propose that feed-forward layers emulate key-value memories, and provide a set of experiments showing that: (a) keys are correlated with human-interpretable input patterns; (b) values, mostly in the model's upper layers, induce distributions over the output vocabulary that correlate with the next-token distribution of patterns in the corresponding key; and (c) the model's output is formed via an aggregation of these distributions, whereby they are first composed to form individual layer outputs, which are then refined throughout the model's layers using residual connections.

Our findings open important research directions:

- **Layer embedding space.** We observe a correlation between value distributions over the output